k8s-01 tasks

1. Setup Minikube in your local machine..

K8s-01 – Task 1: Setup Minikube on Local Windows Machine

1. Objective

The objective of this task is to install and configure Minikube on a local Windows machine and verify that the Kubernetes cluster is running successfully.

2. What is Minikube?

Minikube is a tool that allows us to run a Kubernetes cluster locally on our computer.

It is mainly used for:

- Learning Kubernetes.
- Practising Kubernetes commands.
- Deploying applications such as Nginx.
- Understanding Kubernetes Pods, Deployments and Services.

Simple explanation: Minikube creates a small Kubernetes environment on our local machine without requiring multiple physical servers.

3. Environment Details

Component	Version / Details
Operating System	Windows 11

Driver	Docker
Docker Desktop	4.91.0
Docker Engine	29.8.0
Minikube	v1.39.0
kubectl Client	v1.36.1
Kubernetes Server	v1.37.0
Container Runtime	containerd v2.3.4
Node Operating System	Debian GNU/Linux 12
Node Name	minikube
Node IP	192.168.49.2

4. Prerequisites

Before installing Minikube, we need:

1. Windows 64-bit operating system.
2. Docker Desktop installed and running (for Docker driver).
3. Minikube installed.

4. kubectl installed.
5. Internet connection to download Kubernetes images.
6. Sufficient CPU and RAM resources. A practical starting point is 2 CPUs and 4 GB RAM.

5. Step-by-Step Procedure

Step 1: Verify System Requirements

Command:

```
None
systeminfo
```

Verification:

- Operating System: Windows 11
- RAM: 16 GB
- Virtualization: Hypervisor detected

Purpose:

Before installing Minikube, verify that the computer has sufficient resources and supports the required virtualization environment.

Screenshot 1: Capture the Windows system information showing OS, memory and virtualization details.

Step 2: Verify Docker Installation

Command:

```
None
docker version
```

Output verification:

- Docker Desktop: 4.91.0
- Docker Engine: 29.8.0

Purpose:

Minikube uses Docker as its driver to create and run the local Kubernetes node.

Architecture:

```
None
Windows 11

    |

Docker Desktop

    |

Docker Driver

    |

Minikube Node

    |

Kubernetes Cluster
```

Note: Docker is the Minikube driver in this setup. The Kubernetes node uses containerd as its container runtime.

Screenshot 2: Capture the Docker version output.

Step 3: Verify Minikube Availability

Command:

```
None
minikube version
```

Initial result:

```
None
minikube: command not found
```

Analysis:

Minikube was not available through the Windows command line at this stage.

Step 4: Search for the Minikube Package

Command:

```
None
winget search minikube
```

Output:

```
None
Kubernetes.minikube

Version 1.39.0
```

Purpose:

Search for the Minikube package using Windows Package Manager (Winget).

Step 5: Install Minikube

Command:

```
None
winget install --id Kubernetes.minikube -e
```

Installation output:

```
None
Downloading minikube-installer.exe


Successfully verified installer hash


Successfully installed
```

Purpose:

Install Minikube using Windows Package Manager.

Step 6: Verify Minikube Installation Location

Command:

```
None
Get-ChildItem "C:\Program Files" -Recurse -Filter minikube.exe
```

Output:

```
None
C:\Program Files\Kubernetes\Minikube\minikube.exe
```

Observation:

The Minikube executable was successfully installed in the specified directory.

Step 7: Verify Minikube Command

Command:

```
None
minikube version
```

Initial output:

```
None
The term 'minikube' is not recognized
```

Root Cause:

The Minikube installation directory was not added to the Windows PATH environment variable.

Step 8: Configure the PATH Environment Variable

Procedure:

1. Open Windows Search.
2. Search for `Edit the system environment variables`.
3. Open Environment Variables.
4. Select the Path variable.
5. Click Edit.
6. Click New.
7. Add the following path:

```
None
C:\Program Files\Kubernetes\Minikube
```

8. Click OK to save the changes.

Purpose:

Adding Minikube to PATH allows Windows to recognize and execute the Minikube command from any terminal location.

Step 9: Restart the Terminal

Close the existing PowerShell window and open a new PowerShell session.

Purpose:

A new terminal session loads the updated PATH environment variable.

Step 10: Verify Minikube Installation

Command:

```
None
minikube version
```

Actual output:

```
None
minikube version: v1.39.0

commit: 7a9f6a841470a207de8cf4bafcccee0969d8ba10
```

Purpose:

Confirm that Minikube is installed successfully and the PATH configuration is working.

Step 11: Start the Minikube Cluster

Command:

```
None
minikube start --driver=docker
```

Purpose:

- Start the local Kubernetes cluster.
- Use Docker as the Minikube driver.
- Create or start the Minikube control-plane node.
- Start the Kubernetes components.

Note: If the cluster is already running, this command can reuse the existing Minikube profile.

Step 12: Verify Minikube Cluster Status

Command:

```
None
minikube status
```

Actual output:

```
None
minikube

type: Control Plane

host: Running

kubelet: Running

apiserver: Running

kubeconfig: Configured
```

Purpose:

Verify that the Minikube node and essential Kubernetes services are running.

Step 13: Verify kubectl Installation

Command:

```
None
kubectl version --client
```

Actual output:

```
None
Client Version: v1.36.1

Kustomize Version: v5.8.1
```

Purpose:

kubectl is the command-line tool used to communicate with and manage Kubernetes clusters.

Step 14: Verify Kubernetes Nodes

Command:

```
None
kubectl get nodes -o wide
```

Actual output:

```
None
NAME       STATUS   ROLES           VERSION   INTERNAL-IP

minikube   Ready    control-plane   v1.37.0   192.168.49.2
```

Purpose:

Verify that the Kubernetes node is successfully created and is in Ready status.

Our local Minikube cluster currently contains one node. It acts as the control plane and also runs application workloads.

Step 15: Verify All Kubernetes Pods

Command:

```
None
kubectl get pods -A
```

Actual output:

```
None
NAMESPACE     NAME                               READY   STATUS
```

```
default       nginx                                1/1     Running
kube-system   coredns-559f6c778d-bg2rh             1/1     Running
kube-system   etcd-minikube                        1/1     Running
kube-system   kindnet-vffk2                         1/1     Running
kube-system   kube-apiserver-minikube              1/1     Running
kube-system   kube-controller-manager-minikube     1/1     Running
kube-system   kube-proxy-gcptn                     1/1     Running
kube-system   kube-scheduler-minikube              1/1     Running
kube-system   storage-provisioner                  1/1     Running
```

Purpose:

Verify that the Kubernetes system Pods are running successfully.

Important components:

- CoreDNS: Provides DNS resolution.
- etcd: Stores Kubernetes cluster information.
- kube-apiserver: Handles Kubernetes API requests.
- kube-controller-manager: Manages Kubernetes controllers.
- kube-scheduler: Selects nodes for scheduling Pods.
- kube-proxy: Helps manage Service networking.
- Storage provisioner: Supports dynamic storage provisioning.

6. Final Verification

Verification	Result

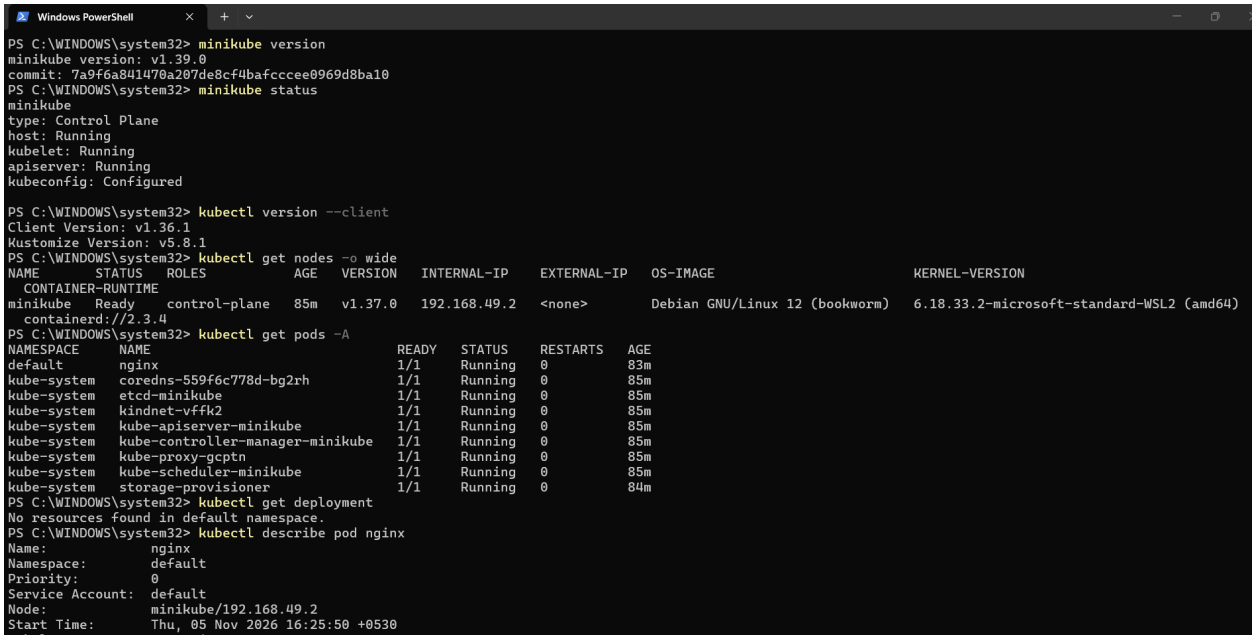
Minikube installation	Successful
Docker driver	Working
Minikube cluster	Running
Kubelet	Running
API Server	Running
Kubeconfig	Configured
kubectl installation	Successful
Kubernetes node	Ready
Kubernetes system Pods	Running

7. Conclusion

Minikube was successfully installed and configured on the local Windows machine using Docker as the driver.

The Kubernetes cluster is running successfully with one control-plane node.

The Kubernetes node is in Ready status, and all essential Kubernetes system Pods are running successfully.



Therefore, **K8s-01 Task 1 – Setup Minikube on Local Machine is completed successfull**.

2. Setup k8s master and two worker nodes on ubuntu..

K8s-01 – Task 2: Setup Kubernetes Master and Two Worker Nodes on Ubuntu

1. Objective

The objective of this task is to create a Kubernetes cluster using three Ubuntu EC2 instances on AWS.

The cluster consists of:

- One Kubernetes Control Plane (Master) node.
- Two Kubernetes Worker nodes.

We use kubeadm to initialize and configure the Kubernetes cluster.

We use containerd as the container runtime and Flannel as the Container Network Interface (CNI) plugin.

2. What is Kubernetes?

Kubernetes is an open-source container orchestration platform used to automate the deployment, scaling and management of containerized applications.

Simple explanation: Kubernetes helps us manage applications running inside containers across multiple servers.

3. Kubernetes Cluster Architecture

Our Kubernetes cluster contains three EC2 instances.

```
None
                AWS CLOUD
                    |
              Kubernetes Cluster
                    |
          +----------------------+
          |   Control Plane      |
          |     k8s-master       |
          |   172.31.1.229       |
          +----------------------+
                    |
          +----------+-----------+
          |                      |
  +----------------+     +----------------+
  |  Worker Node 1 |     |  Worker Node 2 |
  | k8s-worker-1   |     | k8s-worker-2   |
  | 172.31.2.208   |     | 172.31.1.91    |
  +----------------+     +----------------+
```

4. AWS Environment Details

Component	Configuration
Cloud Provider	AWS
Service	Amazon EC2
Region	US East (N. Virginia)

Availability Zone	us-east-1a
Operating System	Ubuntu 24.04.5 LTS
Instance Type	c7i-flex.large
CPU	2 vCPU
RAM	4 GB
Kubernetes Installation	kubeadm
Kubernetes Version	v1.37.2
Container Runtime	containerd v2.2.1
Pod Network	Flannel
Pod Network CIDR	10.244.0.0/16

EC2 Instance Details

Node	Hostname	Private IP
Master	k8s-master	172.31.1.229
Worker 1	k8s-worker-1	172.31.2.208
Worker 2	k8s-worker-2	172.31.1.91

5. Prerequisites

Before creating the Kubernetes cluster, we need:

1. An AWS account.
2. Three Ubuntu EC2 instances.
3. SSH or EC2 Instance Connect access.
4. Minimum 2 CPU and 2 GB RAM per node; our lab uses 2 vCPU and 4 GB RAM.
5. Internet connectivity for downloading packages and container images.
6. Network connectivity between all three instances.

6. AWS Security Group Configuration

We configured the EC2 security group to allow communication between the Kubernetes nodes.

Required access

Protocol / Port	Purpose
TCP 22	SSH access
TCP 6443	Kubernetes API Server
TCP 10250	Kubelet API
UDP 8472	Flannel VXLAN networking

For our lab, we also configured an inbound rule allowing communication from the same security group between the three EC2 instances.

SSH access was restricted to the user's IP address.

Important: Do not expose all ports to the public internet. In production, use specific security-group rules and restrict access appropriately.

7. Step-by-Step Kubernetes Installation

Step 1: Connect to All Three Ubuntu Instances

Connect to the following EC2 instances using EC2 Instance Connect or SSH.

Master node:

```
None
ssh ubuntu@172.31.1.229
```

Worker node 1:

```
None
ssh ubuntu@172.31.2.208
```

Worker node 2:

```
None
ssh ubuntu@172.31.1.91
```

Note: The private IP addresses are used for communication within the AWS VPC. For SSH from outside the VPC, use the appropriate public IP or EC2 Instance Connect.

Step 2: Update Ubuntu Packages

Run the following command on all three nodes:

```
None
sudo apt update
sudo apt upgrade -y
```

Purpose:

- Refresh the Ubuntu package repository information.
- Install available package updates.
- Prepare the operating system for Kubernetes installation.

Step 3: Disable Swap

Run on all three nodes:

```
None
sudo swapoff -a
```

Verify:

```
None
free -h
```

Purpose:

Kubernetes kubelet requires swap to be disabled by default in our kubeadm configuration.

Disabling swap ensures that Kubernetes can manage memory resources as expected.

Step 4: Load Required Kernel Modules

Run on all three nodes:

```
None
sudo modprobe overlay
sudo modprobe br_netfilter
```

Create the configuration file:

```
None
sudo tee /etc/modules-load.d/k8s.conf <<EOF
overlay
br_netfilter
EOF
```

Purpose:

- overlay: Supports overlay filesystem operations used by container runtimes.
- br_netfilter: Allows bridged network traffic to be processed by netfilter rules.

Step 5: Configure Network Settings

Run on all three nodes:

```
None
sudo tee /etc/sysctl.d/k8s.conf <<EOF
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward = 1
EOF
```

Apply the configuration:

```
None
sudo sysctl --system
```

Verify IP forwarding:

```
None
sysctl net.ipv4.ip_forward
```

Expected output:

```
None
net.ipv4.ip_forward = 1
```

Purpose:

These settings allow the Linux kernel to forward network traffic and process bridged traffic correctly for Kubernetes networking.

Step 6: Install Containerd

Run on all three nodes:

```
None
sudo apt install -y containerd
```

Create the containerd configuration directory:

```
None
sudo mkdir -p /etc/containerd
```

Generate the default configuration:

```
None
sudo containerd config default | sudo tee
/etc/containerd/config.toml > /dev/null
```

Configure SystemdCgroup:

```
None
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/'
/etc/containerd/config.toml
```

Restart and enable containerd:

```
None
sudo systemctl restart containerd
sudo systemctl enable containerd
```

Verify:

```
None
sudo systemctl status containerd
```

Purpose:

Containerd is the container runtime responsible for running containers inside Kubernetes nodes.

Kubernetes uses containerd through the Container Runtime Interface (CRI).

Step 7: Add the Kubernetes Package Repository

Run on all three nodes:

```
None
sudo apt-get update
sudo apt-get install -y apt-transport-https ca-certificates curl
gpg
```

Create the repository key directory:

```
None
sudo mkdir -p -m 755 /etc/apt/keyrings
```

Download the Kubernetes signing key:

```
None
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.37/deb/Release.key
| sudo gpg --dearmor -o
/etc/apt/keyrings/kubernetes-apt-keyring.gpg
```

Add the Kubernetes repository:

```
None
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.37/deb/ /' | sudo tee /etc/apt/sources.list.d/kubernetes.list
```

Update package information:

```
None
sudo apt-get update
```

Purpose:

Configure the official Kubernetes package repository to install kubeadm, kubelet and kubectl.

Step 8: Install Kubernetes Components

Run on all three nodes:

```
None
sudo apt-get install -y kubelet kubeadm kubectl
```

Hold the installed versions:

```
None
sudo apt-mark hold kubelet kubeadm kubectl
```

Verify:

```
None
kubeadm version
kubelet --version
kubectl version --client
```

Actual installed version:

```
None
kubeadm: v1.37.2
kubelet: v1.37.2
kubectl: v1.37.2
```

Verify held packages:

```
None
apt-mark showhold
```

Expected:

```
None
kubeadm
kubectl
kubelet
```

Purpose of each component:

- kubeadm: Initializes and configures the Kubernetes cluster.
- kubelet: Runs on every node and communicates with the control plane.
- kubectl: Command-line tool used to manage Kubernetes resources.

Step 9: Initialize the Kubernetes Control Plane

Perform this step only on the master node.

Connect to:

```
None
k8s-master
172.31.1.229
```

Execute:

```
None
sudo kubeadm init \
  --apiserver-advertise-address=172.31.1.229 \
```

```
  --pod-network-cidr=10.244.0.0/16
```

Purpose:

This command initializes the Kubernetes control plane.

Important options:

- --apiserver-advertise-address: Specifies the master node's private IP address.
- --pod-network-cidr: Specifies the network range used by Kubernetes Pods.

After successful initialization, kubeadm configures the control-plane components and generates the worker join command.

Step 10: Configure kubectl on the Master Node

Run on the master:

```
None
mkdir -p $HOME/.kube
```

Copy the administrator configuration:

```
None
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
```

Set ownership:

```
None
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Verify:

```
None
kubectl get nodes
```

Purpose:

This configuration allows the Ubuntu user to execute kubectl commands without using sudo each time.

Step 11: Install Flannel Network Plugin

Run on the master node:

```
None
kubectl apply -f
https://github.com/flannel-io/flannel/releases/latest/download/kube
-flannel.yml
```

Verify:

```
None
kubectl get pods -n kube-flannel
```

Purpose:

Flannel is a Container Network Interface (CNI) plugin.

It provides networking between Pods running on different Kubernetes nodes.

Our Pod network CIDR is:

```
None
10.244.0.0/16
```

This matches the CIDR specified during kubeadm initialization.

Step 12: Generate the Worker Join Command

Run on the master node:

```
None
sudo kubeadm token create --print-join-command
```

This generates a command similar to:

```
None
sudo kubeadm join <MASTER-PRIVATE-IP>:6443 \
  --token <TOKEN> \
  --discovery-token-ca-cert-hash sha256:<HASH>
```

Purpose:

The join command allows worker nodes to securely register with the Kubernetes control plane.

Important: The actual token is sensitive. Do not publish it in documentation or screenshots. Generate a fresh token if required.

Step 13: Join Worker Node 1

Connect to:

```
None
k8s-worker-1
172.31.2.208
```

Execute the actual join command generated on the master:

```
None
sudo kubeadm join <MASTER-PRIVATE-IP>:6443 \
  --token <TOKEN> \
  --discovery-token-ca-cert-hash sha256:<HASH>
```

Expected success message:

```
None
This node has joined the cluster.
```

Step 14: Join Worker Node 2

Connect to:

```
None
k8s-worker-2
```

```
172.31.1.91
```

Execute the same valid join command generated on the master, while its token remains valid.

Expected success message:

```
None
This node has joined the cluster.
```

Purpose:

Both worker nodes register themselves with the Kubernetes control plane.

The control plane can now schedule application Pods on the worker nodes.

Step 15: Verify the Kubernetes Cluster

Return to the master node.

Execute:

```
None
kubectl get nodes -o wide
```

Actual verification:

```
None
NAME           STATUS   ROLES           VERSION   INTERNAL-IP
k8s-master     Ready    control-plane   v1.37.2   172.31.1.229
k8s-worker-1   Ready    <none>          v1.37.2   172.31.2.208
k8s-worker-2   Ready    <none>          v1.37.2   172.31.1.91
```

Explanation:

- k8s-master: Kubernetes control-plane node.
- k8s-worker-1: First worker node.
- k8s-worker-2: Second worker node.
- Ready: Node is healthy and available for scheduling.

The <none> role for worker nodes is normal. It does not mean that they failed to join.

8. Kubernetes Cluster Components

Control Plane Components

1. kube-apiserver – Entry point for Kubernetes API requests.
2. etcd – Stores Kubernetes cluster data.
3. kube-scheduler – Selects a suitable node for newly created Pods.
4. kube-controller-manager – Runs controllers that maintain the desired cluster state.

Worker Node Components

1. kubelet – Communicates with the control plane and manages Pods on the node.
2. kube-proxy – Implements Kubernetes Service networking rules.
3. containerd – Runs application containers.
4. Flannel (CNI) – Provides Pod networking across nodes.

9. Final Verification

Verification	Result
Three Ubuntu EC2 instances	Successful
Swap disabled	Successful
Kernel modules loaded	Successful
Network configuration	Successful
Containerd installed	Successful
Kubernetes packages installed	Successful
Control plane initialized	Successful
Flannel installed	Successful

Worker 1 joined	Successful
Worker 2 joined	Successful
All three nodes Ready	Successful

10. Conclusion

Successfully created a three-node Kubernetes cluster on AWS EC2 using Ubuntu 24.04.

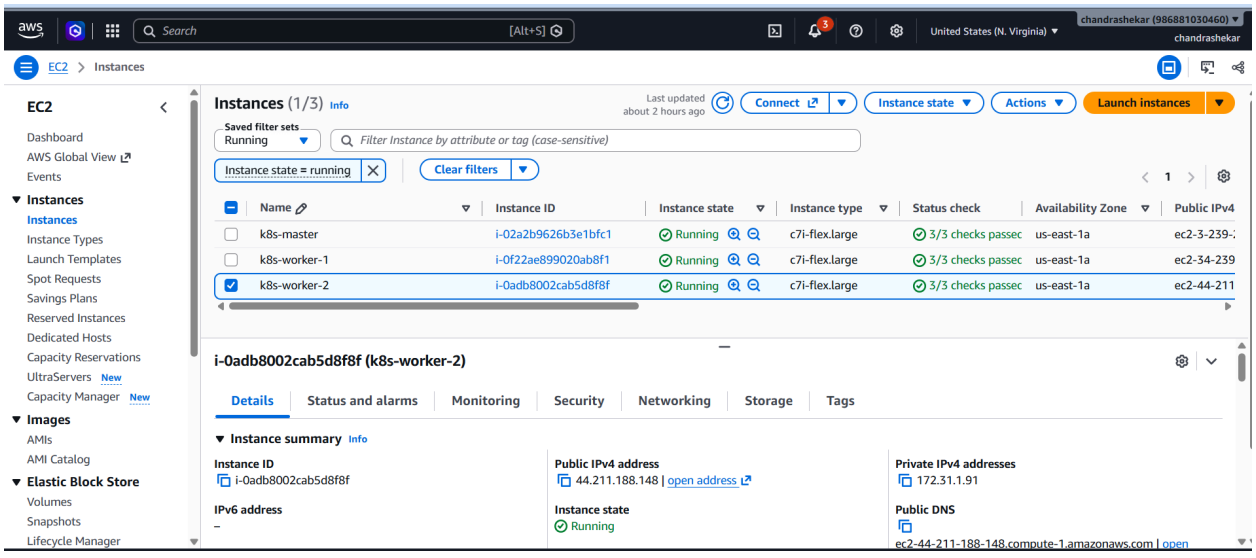
One control-plane node and two worker nodes were configured using kubeadm.

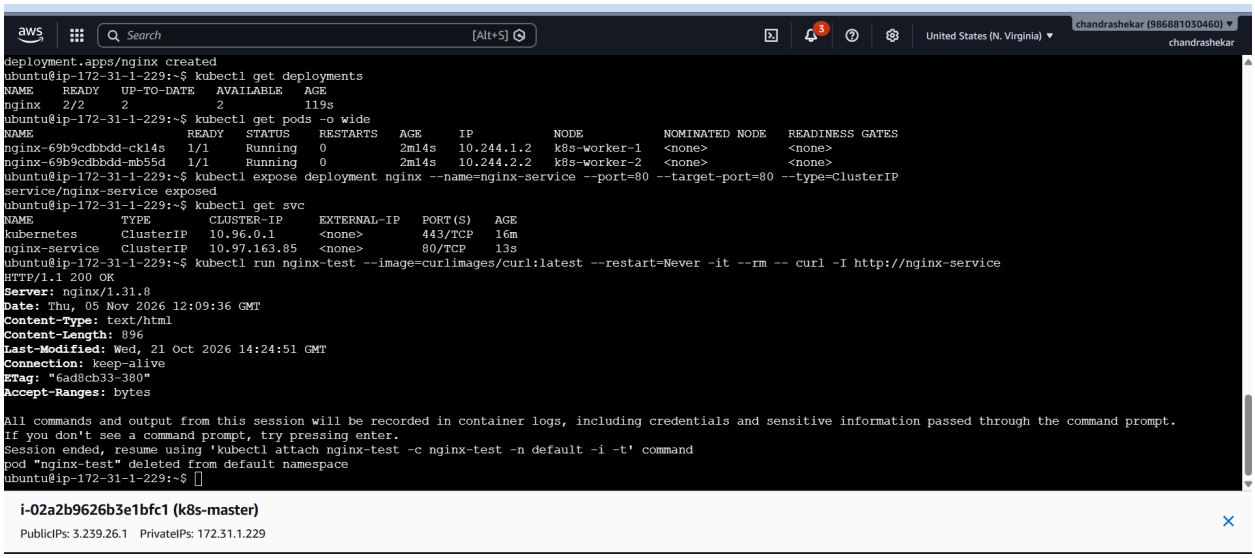
Containerd was installed as the container runtime, and Flannel was configured as the CNI network plugin.

Both worker nodes successfully joined the control plane.

Finally, all three nodes were verified in Ready status using kubectl.

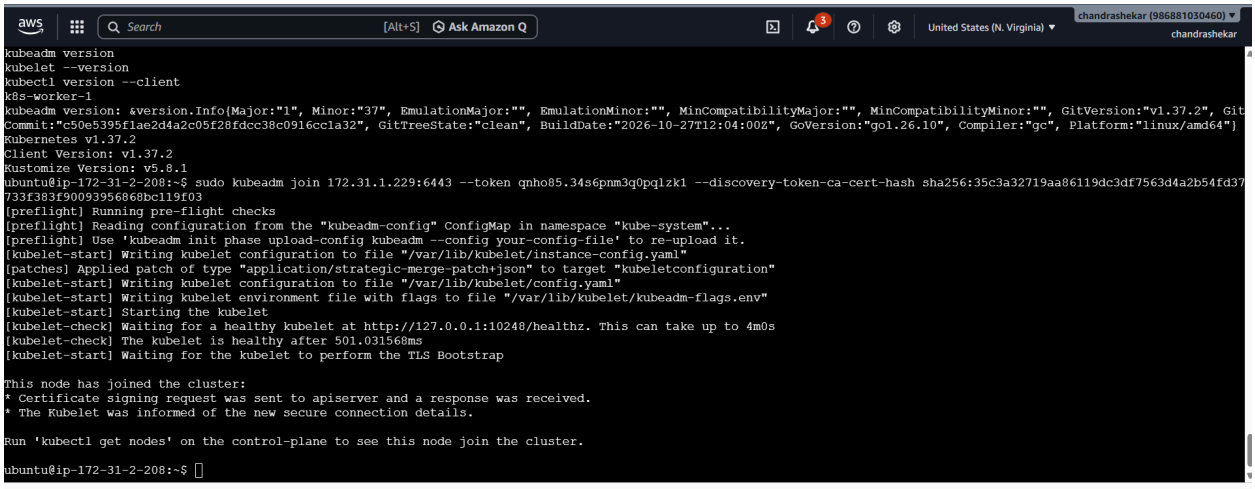
Therefore, **K8s-01 Task 2 – Setup Kubernetes Master and Two Worker Nodes on Ubuntu is completed successfully.**





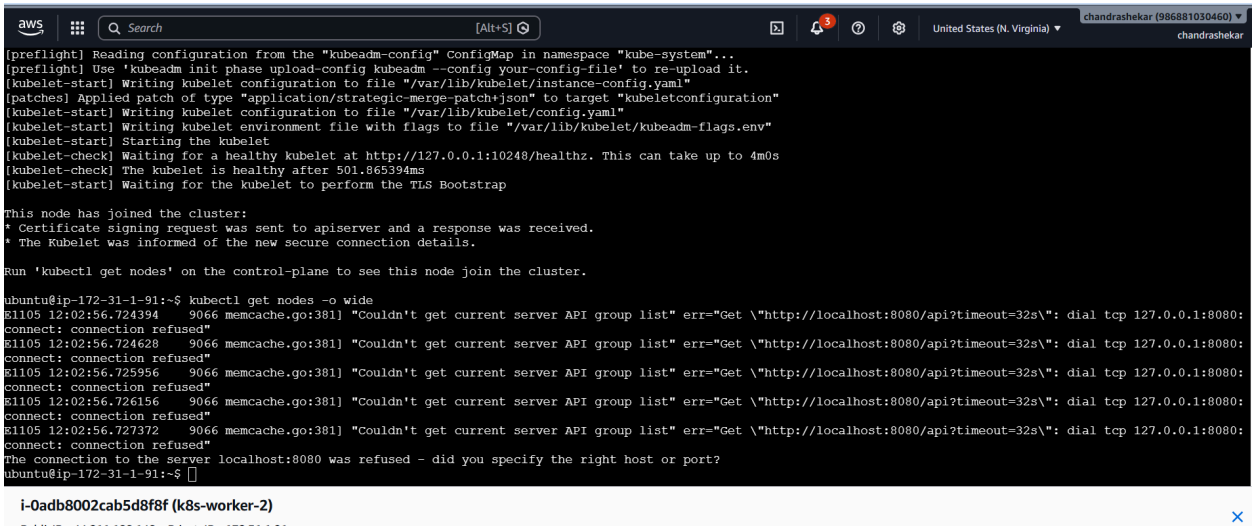
i-02a2b9626b3e1bfc1 (k8s-master)

PublicIPs: 3.239.26.1 PrivateIPs: 172.31.1.229



i-0f22ae899020ab8f1 (k8s-worker-1)

PublicIPs: 34.239.240.121 PrivateIPs: 172.31.2.208



i-0adb8002cab5d8f8f (k8s-worker-2)

PublicIPs: 44.211.188.148 PrivateIPs: 172.31.1.91

3. Run one nginx pod..

TASK 3 – Deploy and Run an Nginx Application in Kubernetes

1. Objective

The objective of this task is to deploy an Nginx application in Kubernetes, verify that the Pods are running successfully, expose the application using Kubernetes Services, and access the Nginx welcome page through a web browser.

2. Environment Details

Component	Details
Operating System	Windows 11
Kubernetes Environment	Minikube
Minikube Driver	Docker
Application	Nginx
Container Image	nginx
Deployment Name	nginx
Number of Replicas	2
Service Types	ClusterIP and NodePort

3. What is Nginx?

Nginx is a web server that can also work as a reverse proxy and load balancer.

In this task, we use the Nginx container image to deploy a simple web application in Kubernetes.

4. Step-by-Step Implementation

Step 1: Verify the Kubernetes Cluster

First, check whether the Minikube cluster is running.

Command:

```
None
minikube status
```

This command checks the status of Minikube components.

Expected result: Host, kubelet, and API server should be running.

Step 2: Check Kubernetes Nodes

Command:

```
None
kubectl get nodes -o wide
```

This command displays the Kubernetes nodes and their status.

Expected result: The Minikube node should show `Ready`.

Step 3: Check Existing Pods

Command:

```
None
kubectl get pods -A
```

This command displays Pods from all Kubernetes namespaces.

During this task, the Nginx Pod was initially created as a standalone Pod.

Step 4: Create a Standalone Nginx Pod

Command:

```
None
kubectl run nginx --image=nginx
```

Explanation:

- `kubectl run` – Creates a Pod.
- `nginx` – Name of the Pod.
- `--image=nginx` – Uses the official Nginx container image.

Verify the Pod:

```
None
kubectl get pods
```

Check detailed information:

```
None
kubectl describe pod nginx
```

The Pod was running successfully.

Step 5: Understand the Difference Between a Pod and Deployment

Initially, we created a standalone Pod using `kubectl run`.

A standalone Pod is not managed by a Deployment. If it is deleted, Kubernetes does not automatically recreate it.

A Deployment manages Pods and helps maintain the required number of replicas.

For this reason, we moved to a Deployment for the application.

Step 6: Delete the Standalone Pod

Command:

```
None
kubectl delete pod nginx
```

This deletes the standalone Nginx Pod.

Step 7: Create an Nginx Deployment

Command:

```
None
kubectl create deployment nginx --image=nginx --replicas=2
```

Explanation:

- `create deployment` – Creates a Kubernetes Deployment.
- `nginx` – Deployment name.
- `--image=nginx` – Uses the Nginx image.
- `--replicas=2` – Requests two Nginx Pods.

Verify the Deployment:

```
None
kubectl get deployments
```

Verify the Pods:

```
None
kubectl get pods -o wide
```

Actual result: The Deployment showed 2/2 ready replicas, and both Nginx Pods were in the Running state.

Step 8: Expose the Deployment Using ClusterIP

A ClusterIP Service allows communication with the application from inside the Kubernetes cluster.

Command:

```
None
kubectl expose deployment nginx --name=nginx-service --port=80
--target-port=80 --type=ClusterIP
```

Explanation:

- `expose deployment nginx` – Exposes the Nginx Deployment.
- `--name=nginx-service` – Service name.
- `--port=80` – Service port.

- `--target-port=80` – Container port.
- `--type=ClusterIP` – Creates an internal Kubernetes Service.

Verify the Service:

```
None
kubectl get services
```

Check Service endpoints:

```
None
kubectl get endpoints nginx-service
```

The Service was assigned ClusterIP `10.109.206.157`, and its endpoints pointed to the two Nginx Pods.

Step 9: Test Nginx Using the ClusterIP Service

To test internal connectivity, we created a temporary curl Pod.

Command:

```
None
kubectl run nginx-test --image=curlimages/curl:latest
--restart=Never -it --rm -- curl -I http://nginx-service
```

Explanation:

- `curlimages/curl` – Container image containing curl.
- `--restart=Never` – Creates a Pod instead of a Deployment.
- `-it` – Runs interactively.
- `--rm` – Removes the temporary Pod after execution.
- `curl -I` – Requests HTTP response headers.

Actual result:

```
None
HTTP/1.1 200 OK
Server: nginx/1.31.8
```

This confirms that the Nginx application was accessible through the internal ClusterIP Service.

Step 10: Expose Nginx Using NodePort

A NodePort Service exposes an application through a port on the Kubernetes node.

Command:

```
None
kubectl expose deployment nginx --name=nginx-nodeport --port=80
--target-port=80 --type=NodePort
```

Verify Services:

```
None
kubectl get services
```

Actual result: The NodePort Service was assigned port `30380`.

The NodePort allows access to the application through the node's address and the assigned port, subject to the environment's networking configuration.

Step 11: Get the Minikube IP Address

Command:

```
None
minikube ip
```

Actual result:

```
None
192.168.49.2
```

This is the Minikube node IP address.

Step 12: Access Nginx Through the Browser

Because Minikube was running with the Docker driver on Windows, we used the Minikube Service command to create a local tunnel.

Command:

```
None
minikube service nginx-nodeport --url
```

Actual result: Minikube generated a local URL similar to:

```
None
http://127.0.0.1:50241
```

We opened the generated URL in a web browser.

The browser displayed the **Welcome to nginx!** page successfully.

Note: Keep the terminal running while using the generated URL. The local tunnel must remain active.

5. Final Verification Commands

Run the following commands to verify the application:

```
None
kubectl get deployments
kubectl get pods -o wide
kubectl get services
kubectl get endpoints nginx-service
```

Final result:

- Nginx Deployment created successfully.
- Two Nginx replicas were running.
- ClusterIP Service created successfully.
- Internal connectivity test returned HTTP 200 OK.
- NodePort Service created successfully.
- Nginx welcome page opened successfully in the browser.

6. Kubernetes Concepts Learned

Concept	Explanation

Pod	Smallest deployable unit in Kubernetes
Deployment	Manages Pods and maintains the desired replica count
Replica	A copy of an application Pod
ClusterIP	Exposes an application internally within the cluster
NodePort	Exposes an application through a port on a node
Endpoint	Represents the backend Pod addresses associated with a Service
kubectl	Command-line tool used to communicate with Kubernetes

7. Screenshots to Include in the Assignment

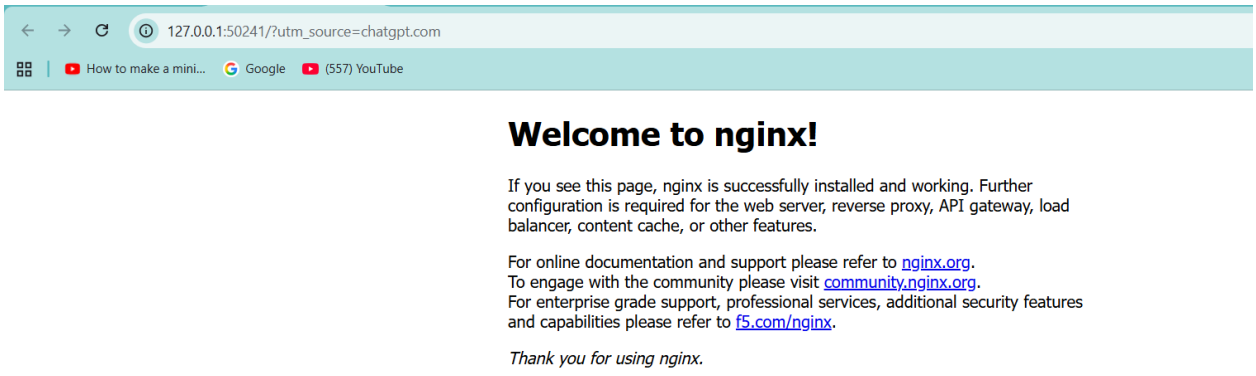
Capture your actual terminal or browser screenshots for these steps:

1. `minikube status`
2. `kubectl get nodes -o wide`
3. `kubectl run nginx --image=nginx`
4. `kubectl describe pod nginx`
5. `kubectl create deployment nginx --image=nginx --replicas=2`
6. `kubectl get deployments`
7. `kubectl get pods -o wide`
8. `kubectl get services`
9. `kubectl get endpoints nginx-service`
10. Curl test showing `HTTP/1.1 200 OK`
11. NodePort Service showing port `30380`
12. Browser displaying the **Welcome to nginx!** page

8. Conclusion

In this task, we successfully deployed an Nginx application in Kubernetes using Minikube.

We learned how to create a standalone Pod, create a Deployment with two replicas, expose the application using ClusterIP and NodePort Services, test internal connectivity using curl, and access the application through a web browser.



Task 3 – Deploy and Run an Nginx Application in Kubernetes: COMPLETED.

4. Mugup Master and slave components on k8s.

K8s-01 – TASK 4: Study Kubernetes Control Plane and Worker Node Components

1. Objective

The objective of this task is to understand and study the major components of Kubernetes Control Plane (Master Node) and Worker Nodes.

We will learn the purpose of each component, how they work together, and how to verify them in our Kubernetes cluster.

2. Kubernetes Architecture Overview

Kubernetes is a container orchestration platform used to deploy, manage, scale, and monitor containerized applications.

A Kubernetes cluster mainly consists of two types of nodes:

1. Control Plane Node (Master)
2. Worker Nodes

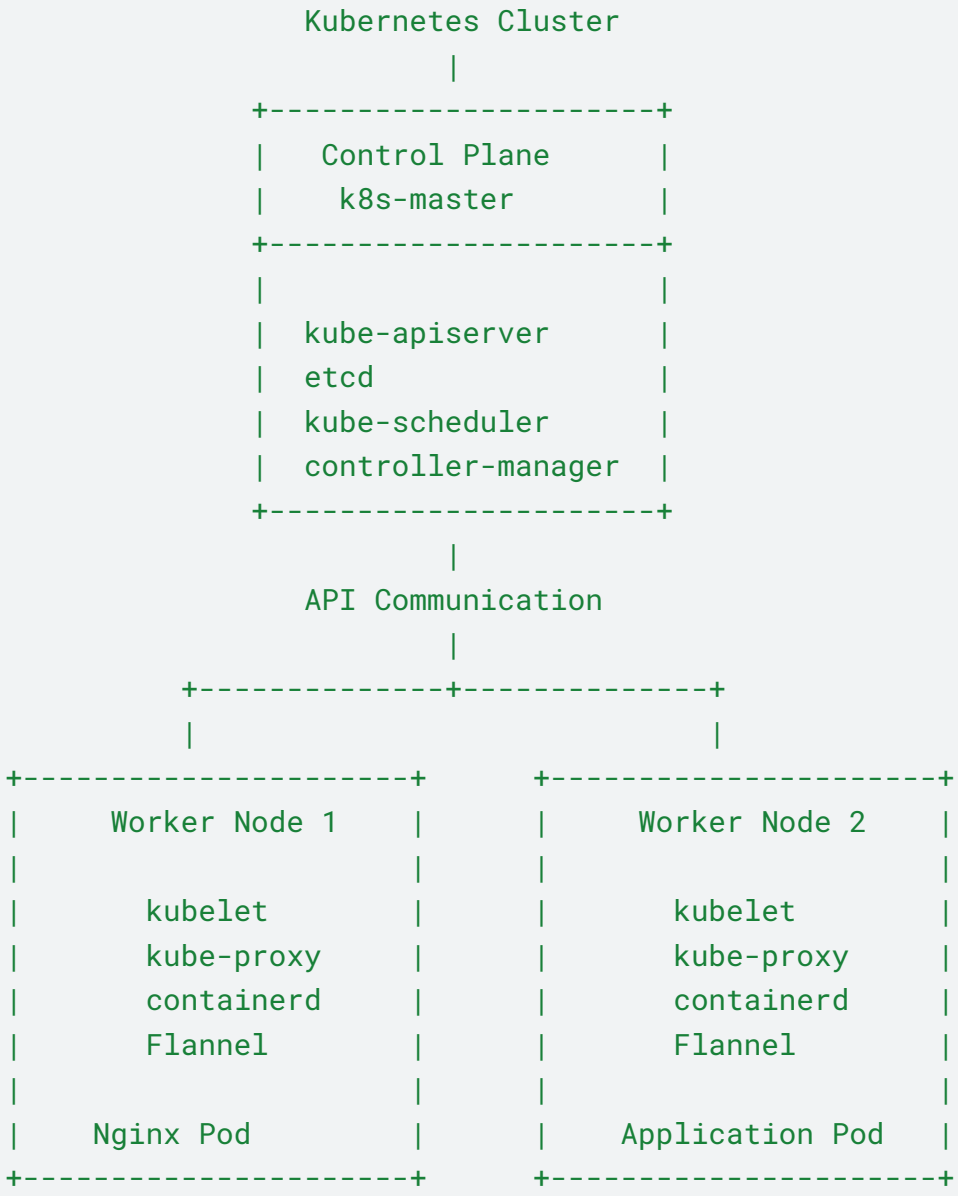
Our AWS EC2 cluster contains:

Node	Private IP	Role

k8s-master	172.31.1.229	Control Plane
k8s-worker-1	172.31.2.208	Worker
k8s-worker-2	172.31.1.91	Worker

3. Kubernetes Architecture Diagram

None



4. Control Plane Components (Master Node)

The Control Plane manages the overall Kubernetes cluster. It makes decisions about scheduling, maintaining the desired state, and responding to cluster events.

There are four major Control Plane components.

4.1 kube-apiserver

What is kube-apiserver?

The kube-apiserver is the front door of the Kubernetes cluster.

It is the main component through which users, administrators, and other Kubernetes components communicate with the cluster.

Responsibilities:

- Receives requests from kubectl.
- Validates API requests.
- Handles authentication and authorization.
- Processes admission control.
- Reads and updates cluster data in etcd.
- Provides the Kubernetes API.

Example:

When we execute:

```
None
kubectl get nodes
```

The kubectl command communicates with the kube-apiserver to retrieve node information.

Interview answer:

The kube-apiserver is the main entry point of Kubernetes. It accepts and processes API requests from kubectl and other components.

4.2 etcd

What is etcd?

etcd is a distributed key-value database used by Kubernetes to store cluster information.

It stores the cluster's persistent state and configuration.

Responsibilities:

- Stores information about Nodes.
- Stores information about Pods.
- Stores Deployments and Services.
- Stores cluster configuration and state.
- Maintains consistent data using distributed consensus.

Example:

When we create a Deployment, Kubernetes stores the relevant cluster state in etcd through the API server.

Interview answer:

etcd is the key-value database of Kubernetes. It stores the cluster's persistent state and configuration.

Important: Always protect and back up etcd because losing its data can affect the Kubernetes cluster.

4.3 kube-scheduler

What is kube-scheduler?

The kube-scheduler is responsible for selecting a suitable Worker Node for newly created Pods that do not yet have a node assigned.

Responsibilities:

- Watches for unscheduled Pods.
- Checks available resources on nodes.
- Considers CPU and memory requirements.
- Considers scheduling rules and constraints.
- Selects a suitable node for the Pod.

Example:

Suppose we create an Nginx Pod.

The scheduler checks the available Worker Nodes and selects a suitable node to run the Pod.

Interview answer:

The kube-scheduler decides which Worker Node should run a newly created Pod based on resource availability and scheduling rules.

4.4 kube-controller-manager

What is kube-controller-manager?

The kube-controller-manager runs different controllers that continuously monitor the cluster and help maintain the desired state.

Responsibilities:

- Monitors Kubernetes resources.
- Checks actual state against desired state.
- Takes corrective action when necessary.
- Manages controllers such as Node Controller and Deployment-related controllers.

Example:

Suppose we create an Nginx Deployment with two replicas.

If one Pod gets deleted, the Deployment controller notices that only one replica is available and creates another Pod to maintain the desired count.

Interview answer:

The kube-controller-manager runs controllers that continuously monitor the cluster and work to maintain the desired state.

5. Worker Node Components

Worker Nodes are responsible for running application workloads in the form of Pods.

There are three important node components, along with the container runtime and network plugin.

5.1 kubelet

What is kubelet?

The kubelet is an agent that runs on every Worker Node.

It communicates with the kube-apiserver and ensures that the Pods assigned to its node are running as expected.

Responsibilities:

- Communicates with the API server.
- Receives Pod specifications.
- Ensures required containers are running.
- Monitors Pod and container health.
- Reports node and Pod status.

Example:

If the scheduler assigns an Nginx Pod to Worker Node 1, the kubelet on Worker Node 1 works with the container runtime to start the required containers.

Interview answer:

The kubelet is an agent running on each node. It ensures that the containers described in the assigned Pod specifications are running properly.

5.2 kube-proxy

What is kube-proxy?

kube-proxy is a network component that helps implement Kubernetes Service networking on nodes.

It maintains network rules that allow traffic to reach the appropriate Service backends.

Responsibilities:

- Helps implement Service networking.
- Routes traffic to Service backend Pods.
- Maintains network rules using mechanisms such as iptables or IPVS, depending on configuration.

Example:

Suppose we have an Nginx application with two Pods and a ClusterIP Service.

When a client accesses the Service, kube-proxy's network rules help direct traffic to one of the backend Pods.

Interview answer:

kube-proxy helps implement Kubernetes Service networking by maintaining network rules that direct traffic to backend Pods.

5.3 Container Runtime – containerd

What is a Container Runtime?

A container runtime is responsible for running containers on a Kubernetes node.

In our AWS Kubernetes cluster, we use containerd.

Responsibilities:

- Pulls container images.
- Creates containers.
- Starts and stops containers.
- Manages the container lifecycle.

Example:

When Kubernetes schedules an Nginx Pod on a Worker Node, kubelet communicates with containerd through the Container Runtime Interface (CRI) to run the container.

Interview answer:

containerd is the container runtime used in our cluster. It pulls images and manages the lifecycle of containers.

5.4 CNI Plugin – Flannel

What is CNI?

CNI stands for Container Network Interface.

It provides the networking functionality needed for communication between Kubernetes Pods.

In our cluster, we installed Flannel as the CNI network plugin.

Responsibilities:

- Provides Pod networking.
- Assigns Pod IP addresses through the configured network setup.
- Helps enable communication between Pods on different nodes.

Example:

An Nginx Pod running on Worker Node 1 should be able to communicate with another Pod running on Worker Node 2 through the cluster network.

Flannel helps provide this Pod network connectivity.

Interview answer:

Flannel is a Kubernetes network plugin that provides Pod networking and enables communication between Pods across different nodes.

6. How All Kubernetes Components Work Together

Let's understand with a simple Nginx Deployment example.

Suppose we execute:

```
None
kubectl create deployment nginx --image=nginx --replicas=2
```

The following process happens:

1. The user runs the kubectl command.
2. kubectl sends the request to kube-apiserver.
3. kube-apiserver validates and processes the request.
4. The desired cluster state is stored in etcd.
5. The Deployment controller works to create the requested Pods.
6. kube-scheduler selects suitable Worker Nodes for the Pods.
7. The kubelet on each selected node receives the Pod specification.
8. kubelet communicates with containerd through CRI.
9. containerd pulls the Nginx image if needed and starts the containers.
10. Flannel provides the Pod network connectivity.
11. kubelet reports Pod status to the API server.
12. We can check the result using kubectl.

7. Practical Commands to Verify Kubernetes Components

Run these commands from the Control Plane node or a machine configured with the cluster's kubectl access.

Check all Nodes

```
None
kubectl get nodes -o wide
```

Check all Pods in every Namespace

```
None
kubectl get pods -A
```

Check Control Plane Pods

```
None
kubectl get pods -n kube-system -o wide
```

Look for components such as:

- kube-apiserver
- kube-controller-manager
- kube-scheduler
- etcd

Check kube-proxy Pods

```
None
kubectl get pods -n kube-system -l k8s-app=kube-proxy
```

Check Flannel Pods

```
None
kubectl get pods -n kube-flannel -o wide
```

Check Node Details

```
None
kubectl describe node k8s-master
kubectl describe node k8s-worker-1
kubectl describe node k8s-worker-2
```

Check containerd on a Worker Node

Connect to a Worker Node and run:

```
None
sudo systemctl status containerd
```

Check kubelet on a Worker Node

```
None
sudo systemctl status kubelet
```

8. Important Differences – Control Plane vs Worker Node

Control Plane	Worker Node
Manages the cluster	Runs application workloads
kube-apiserver	kubelet
etcd	kube-proxy
kube-scheduler	containerd
kube-controller-manager	Flannel CNI agent
Makes cluster management decisions	Runs Pods and containers

9. Important Interview Questions and Answers

Q1. What is the Kubernetes Control Plane?

The Control Plane manages the Kubernetes cluster and makes decisions about scheduling and maintaining the desired state.

Q2. What is the difference between Master and Worker Nodes?

The Control Plane manages the cluster, whereas Worker Nodes run application Pods.

Q3. What is kube-apiserver?

It is the main entry point for Kubernetes API requests.

Q4. What is etcd?

It is the distributed key-value database that stores Kubernetes cluster state.

Q5. What is kube-scheduler?

It selects a suitable Worker Node for newly created, unscheduled Pods.

Q6. What is kube-controller-manager?

It runs controllers that monitor the cluster and maintain the desired state.

Q7. What is kubelet?

It is an agent running on each node that ensures assigned Pods are running as expected.

Q8. What is kube-proxy?

It helps implement Kubernetes Service networking and directs traffic to backend Pods.

Q9. What is containerd?

It is a container runtime that manages the lifecycle of containers.

Q10. What is Flannel?

Flannel is a CNI network plugin that provides Pod networking across Kubernetes nodes.

Q11. What happens if a Pod is deleted?

If the Pod is managed by a Deployment, the Deployment controller works to create a replacement Pod to maintain the desired replica count.

Q12. What happens if the kubelet stops on a Worker Node?

The node may become NotReady if it cannot communicate with the Control Plane. Existing containers may continue running for some time, but Kubernetes loses normal kubelet management and status reporting on that node.

Q13. Can we run application Pods on the Control Plane node?

By default, kubeadm configures a control-plane taint that prevents ordinary application Pods from being scheduled there. This can be changed if required.

10. Conclusion

In this task, we studied the important components of Kubernetes Control Plane and Worker Nodes.

We understood the responsibilities of kube-apiserver, etcd, kube-scheduler, kube-controller-manager, kubelet, kube-proxy, containerd, and Flannel.

We also learned how these components work together when deploying an application such as Nginx.

Task 4 – Kubernetes Control Plane and Worker Node Components: COMPLETED.